

Índice

Stockly — Sistema de gestión y reservas de almacén	3
Portada	3
Resumen	3
Abstract	4
Palabras clave	4
Índice	4
1. Introducción	7
1.1 Contexto del proyecto	7
1.2 Objetivos del proyecto	7
1.3 Motivación	8
1.4 Tecnologías utilizadas	8
2. Análisis y diseño del sistema	9
2.1 Descripción general del proyecto	9
2.2 Roles de usuario	9
2.3 Requisitos funcionales	10
2.4 Requisitos no funcionales	11
2.5 Casos de uso	12
2.6 Diseño de interfaces (Figma)	13
2.7 Arquitectura general del sistema	13
3. Diseño de la base de datos	14
3.1 Modelo entidad-relación	14
3.2 Diseño lógico de tablas	15
3.3 Relaciones y claves	15
3.4 Scripts SQL	16
3.5 Datos de prueba (seed)	16
4. Desarrollo del backend	16
4.1 Arquitectura del backend	16
4.2 Configuración inicial del proyecto	17
4.3 Conexión a la base de datos	17
4.4 Desarrollo de la API REST	17
4.5 CRUD de entidades	18
4.6 Validaciones de datos	18
4.7 Manejo global de errores	18
4.8 Autenticación y autorización	18
4.9 Swagger / OpenAPI	19
4.10 Pruebas de endpoints	19
5. Desarrollo de la aplicación web	20
5.1 Diseño inicial en Figma	20
5.2 Estructura del frontend	20
5.3 Integración con la API REST	20
5.4 Gestión de sesiones y login	20
5.5 CRUD de datos	21

5.6 Funcionalidades según rol	21
5.7 Manejo de errores	21
5.8 Capturas de la aplicación	21
6. Desarrollo de la aplicación móvil	22
6.1 Diseño inicial en Figma	22
6.2 Navegación entre pantallas	22
6.3 Conexión con la API REST	23
6.4 Persistencia local	23
6.5 Gestión de sesiones	24
6.6 Funcionalidades futuras	24
6.7 Capturas de la aplicación	24
7. Despliegue e infraestructura	25
7.1 Variables de entorno	25
7.2 Despliegue del backend	25
7.3 Base de datos remota	26
7.4 Hosting y servicios cloud	26
7.5 Git y GitHub	27
7.6 GitHub Actions / CI/CD	27
7.7 Acceso público al sistema	27
8. Seguridad	27
8.1 Autenticación	27
8.2 Autorización	28
8.3 Protección de datos	28
8.4 Buenas prácticas aplicadas	29
9. Inteligencia Artificial aplicada al proyecto	29
9.1 Herramientas utilizadas	29
9.2 Uso realizado	29
9.3 Validación y revisión	30
10. Pruebas y validación	31
10.1 Pruebas funcionales	31
10.2 Casos de prueba	31
10.3 Resultados obtenidos	32
10.4 Errores encontrados y soluciones	32
11. Conclusiones	33
11.1 Resultados obtenidos	33
11.2 Problemas encontrados	33
11.3 Mejoras futuras	34
11.4 Valoración personal	34
12. Bibliografía	35
13. Anexos	35
Anexo A — Manual de instalación	35

Anexo B — Manual de usuario	36
Anexo C — Enlaces	36
Anexo D — Fragmentos de código relevantes	37
D.1 Reserva con control de concurrencia	37
D.2 Service Worker de autodesregistro	38
D.3 Capa de red Android	39

Stockly — Sistema de gestión y reservas de almacén

Trabajo Fin de Ciclo · Desarrollo de Aplicaciones Multiplataforma (DAM)

Portada

- **Título:** Stockly — Sistema de gestión de inventario y reservas para almacén
 - **Alumnos:** Adrián Bravo Santos y Miguel Ángel Florido
 - **Ciclo formativo:** Desarrollo de Aplicaciones Multiplataforma (DAM)
 - **Centro educativo:** CDM Formación Alcorcón
 - **Curso académico:** 2025/2026
 - **Tutor/a:** Damián Sualdea Soy
 - **Fecha de entrega:** 26/05/2026
-

Resumen

Stockly es una aplicación web para gestionar el inventario y las reservas de un almacén. Muchas empresas pequeñas todavía usan hojas de cálculo para llevar el control del stock, lo que provoca errores cuando dos personas intentan reservar el mismo producto a la vez. Stockly soluciona este problema ofreciendo una plataforma centralizada con tres tipos de usuario (cliente, operario y administrador), un sistema de reservas seguro y una aplicación móvil para Android. La aplicación está publicada en internet y cualquier persona con acceso puede usarla desde el navegador o desde el móvil.

Abstract

Stockly is a web application for managing warehouse inventory and reservations. Many small businesses still use spreadsheets to track stock, which leads to errors when two people try to reserve the same product simultaneously. Stockly solves this by providing a centralized platform with three user roles (client, operator, and administrator), a safe reservation system, and an Android mobile application. The app is published online and can be accessed from any browser or mobile device.

Palabras clave

Aplicación web · Node.js · Base de datos · API REST · Gestión de stock · Reservas · PWA · App Android · Despliegue cloud · Kotlin

Índice

En el documento Word final, sustituir este índice por el generado automáticamente con: Referencias → Tabla de contenido → Tabla automática.

Sección

Resumen

Abstract

Palabras clave

1. Introducción

1.1 Contexto del proyecto

1.2 Objetivos del proyecto

1.3 Motivación

1.4 Tecnologías utilizadas

2. Análisis y diseño del sistema

2.1 Descripción general del proyecto

2.2 Roles de usuario

2.3 Requisitos funcionales

2.4 Requisitos no funcionales

Sección

2.5 Casos de uso

2.6 Diseño de interfaces (Figma)

2.7 Arquitectura general del sistema

3. Diseño de la base de datos

3.1 Modelo entidad-relación

3.2 Diseño lógico de tablas

3.3 Relaciones y claves

3.4 Scripts SQL

3.5 Datos de prueba (seed)

4. Desarrollo del backend

4.1 Arquitectura del backend

4.2 Configuración inicial del proyecto

4.3 Conexión a la base de datos

4.4 Desarrollo de la API REST

4.5 CRUD de entidades

4.6 Validaciones de datos

4.7 Manejo global de errores

4.8 Autenticación y autorización

4.9 Swagger / OpenAPI

4.10 Pruebas de endpoints

5. Desarrollo de la aplicación web

5.1 Diseño inicial en Figma

5.2 Estructura del frontend

5.3 Integración con la API REST

5.4 Gestión de sesiones y login

5.5 CRUD de datos

5.6 Funcionalidades según rol

5.7 Manejo de errores

5.8 Capturas de la aplicación

6. Desarrollo de la aplicación móvil

Sección

- 6.1 Diseño inicial en Figma
- 6.2 Navegación entre pantallas
- 6.3 Conexión con la API REST
- 6.4 Persistencia local
- 6.5 Gestión de sesiones
- 6.6 Funcionalidades futuras
- 6.7 Capturas de la aplicación

7. Despliegue e infraestructura

- 7.1 Variables de entorno
- 7.2 Despliegue del backend
- 7.3 Base de datos remota
- 7.4 Hosting y servicios cloud
- 7.5 Git y GitHub
- 7.6 GitHub Actions / CI/CD
- 7.7 Acceso público al sistema

8. Seguridad

- 8.1 Autenticación
- 8.2 Autorización
- 8.3 Protección de datos
- 8.4 Buenas prácticas aplicadas

9. Inteligencia Artificial aplicada al proyecto

- 9.1 Herramientas utilizadas
- 9.2 Uso realizado
- 9.3 Validación y revisión

10. Pruebas y validación

- 10.1 Pruebas funcionales
- 10.2 Casos de prueba
- 10.3 Resultados obtenidos
- 10.4 Errores encontrados y soluciones

11. Conclusiones

Sección

11.1 Resultados obtenidos

11.2 Problemas encontrados

11.3 Mejoras futuras

11.4 Valoración personal

12. Bibliografía

13. Anexos

Anexo A. Manual de instalación

Anexo B. Manual de usuario

Anexo C. Enlaces

Anexo D. Fragmentos de código relevantes

1. Introducción

1.1 Contexto del proyecto

En muchas empresas pequeñas — tiendas, talleres, almacenes — el control del stock se sigue haciendo con hojas de cálculo compartidas o incluso con papel y bolígrafo. Esto funciona cuando hay pocas personas trabajando, pero en cuanto el equipo crece empiezan los problemas: dos empleados reservan el mismo producto a la vez, nadie sabe quién se llevó qué, y el inventario no refleja la realidad.

Stockly nace para resolver exactamente eso: dar a cualquier almacén pequeño una herramienta sencilla, gratuita y accesible desde cualquier dispositivo para gestionar su stock y sus reservas de forma ordenada.

1.2 Objetivos del proyecto

Objetivo principal: crear una aplicación completa (web y móvil) que permita gestionar el inventario y las reservas de un almacén, con diferentes niveles de acceso según el tipo de usuario, y que esté disponible en internet para que cualquier persona del

equipo pueda usarla desde cualquier lugar.

Objetivos específicos:

- Diseñar una base de datos que almacene toda la información del almacén de forma organizada.
- Crear un servidor que responda a las peticiones de la web y la app móvil.
- Garantizar que dos personas no puedan reservar más stock del que hay, aunque intenten hacerlo al mismo tiempo.
- Desarrollar una página web moderna que funcione también sin conexión y se pueda instalar como una app de escritorio.
- Crear una aplicación Android para que el operario del almacén gestione las reservas desde su móvil.
- Publicar todo el sistema en internet con una dirección web pública.
- Comprobar que todo funciona correctamente con pruebas automáticas.

1.3 Motivación

Queríamos hacer un proyecto que tuviera sentido en el mundo real y que nos permitiera aplicar todo lo aprendido durante el ciclo: base de datos, programación del servidor, desarrollo web, aplicación móvil y despliegue en la nube. Stockly encaja perfectamente porque necesita todas esas piezas a la vez.

Además, el problema que resuelve es concreto y fácil de entender: cualquier persona que haya trabajado en un almacén o una tienda sabe lo frustrante que es encontrarse con el stock incorrecto por culpa de una hoja de cálculo desactualizada.

1.4 Tecnologías utilizadas

Para desarrollar Stockly hemos utilizado las siguientes tecnologías:

- **Servidor (backend):** Node.js con el framework Express. Es el “motor” de la aplicación: recibe todas las peticiones, consulta la base de datos y devuelve los resultados.
- **Página web (frontend):** HTML, CSS y JavaScript puro, sin librerías externas. Esto hace que la aplicación sea más ligera y fácil de mantener.

- **Base de datos:** MySQL 8, donde se almacena toda la información: usuarios, productos, reservas y movimientos de stock.
 - **Nube / publicación:** Railway, una plataforma que nos permite publicar la aplicación en internet de forma sencilla y gratuita para proyectos pequeños.
 - **App Android:** Kotlin con Jetpack Compose, que es el lenguaje y la librería recomendados actualmente por Google para desarrollar aplicaciones Android.
 - **Herramientas:** Git y GitHub para el control de versiones, Figma para el diseño, Vitest para las pruebas automáticas y VS Code como editor de código.
-

2. Análisis y diseño del sistema

2.1 Descripción general del proyecto

Stockly está formado por tres partes que trabajan juntas:

1. **El servidor:** recibe todas las peticiones, gestiona la lógica del negocio (reservas, stock, usuarios) y devuelve los datos necesarios.
2. **La página web:** es la interfaz principal. Se puede usar desde cualquier navegador y también se puede instalar en el ordenador como si fuera una aplicación de escritorio.
3. **La app Android:** pensada específicamente para el operario del almacén. Permite gestionar las reservas directamente desde el móvil sin necesidad de abrir el navegador.

Las tres partes comparten la misma base de datos y el mismo servidor, lo que garantiza que la información siempre esté actualizada y sea consistente.

2.2 Roles de usuario

Stockly tiene tres tipos de usuario, cada uno con diferentes permisos:

Rol	Qué puede hacer
Cliente	Ver el catálogo de productos, hacer reservas y consultar sus propias reservas.
Operario	Todo lo anterior, más confirmar, entregar o cancelar cualquier reserva del almacén.
Administrador	Todo lo anterior, más gestionar productos, categorías y usuarios, ver el panel de estadísticas e importar productos en lote desde un fichero CSV.

El sistema comprueba el tipo de usuario en dos sitios: en el servidor (que es la comprobación real y segura) y en la página web (que simplemente oculta los botones que no corresponden).

2.3 Requisitos funcionales

Los requisitos funcionales son las cosas que la aplicación tiene que ser capaz de hacer:

ID	Qué tiene que hacer la aplicación
RF-01	Permitir registrarse e iniciar sesión con email y contraseña.
RF-02	El registro libre crea usuarios con el rol de cliente por defecto.
RF-03	Mostrar un catálogo de productos con buscador y filtros por categoría o stock disponible.
RF-04	Permitir a un cliente reservar unidades de un producto si hay stock suficiente.
RF-05	Evitar que dos usuarios reserven más stock del disponible aunque lo intenten a la vez.
RF-06	Permitir al operario cambiar el estado de una reserva: pendiente → confirmada → entregada.
RF-07	Permitir al administrador crear, editar y eliminar productos.

ID	Qué tiene que hacer la aplicación
RF-08	Mostrar un panel de estadísticas con datos clave del almacén (productos, reservas, etc.).
RF-09	Registrar automáticamente cada cambio de stock como un movimiento con fecha y usuario.
RF-10	Importar varios productos a la vez desde un archivo CSV.
RF-11	Exportar las reservas filtradas a un archivo CSV.
RF-12	Funcionar sin conexión básica y poder instalarse como aplicación de escritorio (PWA).
RF-13	Generar un albarán imprimible en formato A4 al entregar una reserva.

2.4 Requisitos no funcionales

Los requisitos no funcionales describen cómo tiene que comportarse la aplicación, más allá de lo que hace:

- **Seguridad.** Las contraseñas se almacenan cifradas. El sistema limita los intentos de inicio de sesión para evitar ataques de fuerza bruta. Todas las comunicaciones van cifradas con HTTPS.
- **Rendimiento.** El catálogo de 500 productos carga en menos de 200 milisegundos. La aplicación arranca casi instantáneamente en visitas repetidas gracias al Service Worker.
- **Escalabilidad.** El servidor no guarda información de sesión en memoria; todo lo necesario viaja dentro del token de acceso. Esto facilita añadir más servidores si fuera necesario.
- **Disponibilidad.** Con el despliegue en Railway, el objetivo es que la aplicación esté disponible el 99% del tiempo con copias de seguridad automáticas de la base de datos.
- **Mantenibilidad.** El código está organizado por capas (rutas, lógica, base de

datos) y cuenta con pruebas automáticas sobre las funcionalidades más importantes.

- **Accesibilidad.** Se ha trabajado con buen contraste de colores y elementos enfocables con teclado. La mejora completa según el estándar WCAG AA está planificada como trabajo futuro.

2.5 Casos de uso

Los casos de uso describen las acciones principales que un usuario puede realizar en el sistema:

CU-01 Iniciar sesión. El usuario escribe su email y contraseña. El servidor comprueba que son correctos y, si es así, devuelve un token de acceso. La aplicación guarda ese token y muestra la vista correspondiente al rol del usuario.

CU-02 Hacer una reserva. El cliente entra en la ficha de un producto e indica cuántas unidades quiere reservar. El servidor bloquea temporalmente el registro del producto en la base de datos, comprueba que hay suficiente stock y, si es así, crea la reserva y descuenta el stock reservado. Si otro usuario se adelantó, devuelve un error indicando que no hay suficiente stock.

CU-03 Entregar una reserva. El operario abre la lista de reservas, selecciona una y la marca como entregada. El sistema descuenta el stock definitivamente, cambia el estado de la reserva y permite imprimir el albarán de entrega.

CU-04 Crear un producto. El administrador pulsa el botón de nuevo producto, rellena el formulario (nombre, código, categoría, precio, stock disponible y stock mínimo) y lo guarda. El producto aparece inmediatamente en el catálogo.

CU-05 Importar productos desde CSV. El administrador selecciona un archivo CSV con los datos de los productos. La aplicación muestra una previsualización indicando qué filas son correctas y cuáles tienen errores. Al confirmar, se insertan todos los productos válidos de una sola vez.

Diagrama UML de casos de uso disponible en docs/diagrams/casos-de-uso.drawio.

2.6 Diseño de interfaces (Figma)

Antes de empezar a programar la interfaz, diseñamos las pantallas en Figma. Pasamos por tres versiones hasta llegar al diseño actual:

1. **Primera versión:** un diseño genérico de panel de administración con efectos de cristal translúcido. Nos ayudó a definir la estructura de la información, pero no tenía personalidad propia.
2. **Segunda versión:** intentamos un estilo más industrial: colores metálicos, textura de acero, bordes marcados, cintas de peligro. Funcionaba temáticamente pero resultaba visualmente recargado y cansado a la vista.
3. **Versión actual (v3):** mantuvimos la tipografía condensada y los bordes rectos del estilo industrial, pero eliminamos todos los adornos. El resultado es más limpio, mejor contraste y más agradable de usar a diario.

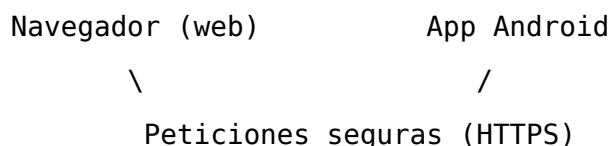
Las pantallas diseñadas son: Login, Registro, Catálogo de productos, Detalle de producto, Mis reservas, Cola de reservas (operario), Panel de estadísticas (administrador), Gestión de inventario y Gestión de usuarios.

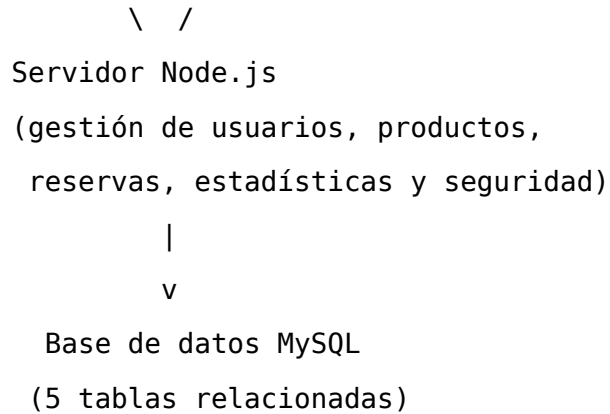
Los mockups de Figma están disponibles en el archivo de diseño del proyecto y se exportarán en imagen para incluirlos en el documento final.

2.7 Arquitectura general del sistema

El sistema sigue una arquitectura cliente-servidor clásica dividida en tres capas:

- **Capa cliente:** la página web (que se ejecuta en el navegador del usuario) y la aplicación Android. Ambas se comunican con el servidor para obtener y enviar datos.
- **Capa servidor:** el programa Node.js que recibe todas las peticiones, comprueba que el usuario tiene permiso, ejecuta la lógica necesaria y responde con los datos.
- **Capa de datos:** la base de datos MySQL donde se almacena toda la información de forma permanente.





Todo el sistema está publicado en Railway, una plataforma en la nube que gestiona automáticamente el certificado de seguridad HTTPS y actualiza la aplicación cada vez que subimos cambios al repositorio de GitHub.

Diagramas detallados disponibles en `docs/diagrams/arquitectura-despliegue.drawio`.

3. Diseño de la base de datos

3.1 Modelo entidad-relación

La base de datos está formada por cinco tablas que representan los datos principales del sistema:

- **usuarios:** almacena los datos de cada persona registrada (nombre, email, contraseña cifrada y tipo de usuario).
- **categorías:** las categorías de los productos del almacén (electrónica, herramientas, etc.).
- **productos:** el catálogo completo con cada artículo, su código, precio, stock disponible y stock reservado.
- **reservas:** cada reserva que hace un cliente, con su estado (pendiente, confirmada, entregada o cancelada) y las fechas correspondientes.
- **movimientos:** un registro automático de cada vez que el stock de un producto cambia, con quién lo cambió y cuándo.

Las relaciones entre tablas son: - Un producto pertenece a una categoría (y si se borra la categoría, el producto queda sin categoría pero no se elimina). - Una reserva está asociada a un usuario y a un producto. - Cada movimiento está asociado a un producto y opcionalmente a un usuario.

Diagrama E/R disponible en docs/diagrams/er.drawio.

3.2 Diseño lógico de tablas

Para el diseño de las tablas hemos aplicado estas decisiones:

- Usamos el juego de caracteres `utf8mb4` para que la base de datos soporte correctamente el español y otros idiomas, incluidos emojis.
- Cada tabla tiene un identificador numérico único que se genera automáticamente al insertar un nuevo registro.
- Todas las tablas guardan la fecha de creación y, donde tiene sentido, la fecha de la última modificación.
- Los estados de las reservas se definen como una lista cerrada de valores permitidos (pendiente, confirmada, cancelada, entregada), lo que evita que se creen datos inválidos.
- Hemos añadido índices en los campos que se consultan con frecuencia (el código de producto, el estado de las reservas) para que las búsquedas sean más rápidas.

3.3 Relaciones y claves

Todas las relaciones entre tablas tienen definida una acción para cuando se borra el registro padre. Por ejemplo, si se borra un producto, todos sus movimientos asociados se borran también. Si se borra una categoría, los productos de esa categoría se quedan sin categoría pero no se eliminan.

El código de cada producto y el email de cada usuario son únicos en toda la base de datos, lo que impide duplicados.

3.4 Scripts SQL

El archivo `db/schema.sql` contiene todo lo necesario para crear la base de datos desde cero: la estructura de todas las tablas, las relaciones entre ellas y los datos iniciales de prueba. Al arrancar la aplicación por primera vez, este archivo se aplica automáticamente si la base de datos está vacía.

3.5 Datos de prueba (seed)

Para facilitar las pruebas, la aplicación incluye datos de ejemplo listos para usar:

- Tres usuarios de prueba con la contraseña `password123`: uno de cada tipo (administrador, operario y cliente).
- Ocho categorías de productos.
- Aproximadamente 500 productos con códigos, nombres, precios y stock realistas.
- Varias reservas en diferentes estados.

Importante: estos usuarios de prueba deben cambiarse o eliminarse antes de usar la aplicación en un entorno real.

4. Desarrollo del backend

4.1 Arquitectura del backend

El servidor está desarrollado con **Node.js** y el framework **Express**, que es una herramienta muy popular para crear servidores web con JavaScript. El código está organizado en carpetas según su función:

- **server.js**: el archivo principal que arranca el servidor y conecta todas las piezas.
- **src/db.js**: gestiona la conexión con la base de datos.
- **src/middleware/auth.js**: comprueba en cada petición si el usuario está identificado y si tiene permiso para lo que quiere hacer.
- **src/routes/**: una carpeta con un archivo por cada grupo de funcionalidades (autenticación, productos, categorías, reservas y administración).

Esta organización hace que el código sea más fácil de entender y mantener: si hay un problema con las reservas, sé exactamente en qué archivo buscar.

4.2 Configuración inicial del proyecto

El servidor necesita saber dónde está la base de datos, qué puerto usar y otras configuraciones que varían entre el ordenador de desarrollo y el servidor en la nube. Toda esta información se almacena en un archivo llamado `.env` que nunca se sube a GitHub por seguridad.

El proyecto incluye un archivo `.env.example` que muestra qué variables hay que configurar, sin revelar los valores reales.

4.3 Conexión a la base de datos

Para conectarse a MySQL utilizamos la librería `mysql2`, que nos permite lanzar varias consultas a la vez de forma eficiente. Todas las consultas usan parámetros separados (en lugar de construir el texto SQL concatenando cadenas), lo que evita uno de los ataques más comunes: la inyección SQL.

4.4 Desarrollo de la API REST

La API es el conjunto de direcciones web a través de las cuales la página web y la app Android se comunican con el servidor. Cada dirección tiene un propósito concreto:

Acción	Dirección	Quién puede usarla
Registrarse /	<code>/api/auth/register</code> y	Cualquiera
Iniciar sesión	<code>/api/auth/login</code>	
Ver mi perfil	<code>/api/auth/me</code>	Usuarios identificados
Ver productos	<code>/api/productos</code>	Usuarios identificados
Crear / editar / borrar productos	<code>/api/productos</code>	Solo administradores
Ver y crear reservas	<code>/api/reservas</code>	Usuarios identificados

Acción	Dirección	Quién puede usarla
Cambiar estado de reservas	/api/reservas/:id	Operarios y administradores
Ver estadísticas	/api/admin/stats	Solo administradores

4.5 CRUD de entidades

Para cada tipo de dato (productos, categorías, reservas, usuarios) hemos implementado las cuatro operaciones básicas:

- **Crear** un nuevo registro.
- **Leer** uno o todos los registros (con filtros y paginación cuando hay muchos).
- **Actualizar** los datos de un registro existente.
- **Eliminar** un registro (o marcarlo como inactivo para no perder el historial).

4.6 Validaciones de datos

Antes de guardar cualquier dato en la base de datos, el servidor comprueba que la información recibida es correcta: que los campos obligatorios no están vacíos, que los números son números, que las cantidades no son negativas, etc. Si algo no cuadra, el servidor devuelve un mensaje de error claro indicando qué está mal.

4.7 Manejo global de errores

Si en algún momento el servidor encuentra un error inesperado (por ejemplo, la base de datos no responde), en lugar de mostrar información técnica que podría ser un riesgo de seguridad, devuelve un mensaje de error genérico al usuario. Los detalles del error solo aparecen en los registros internos del servidor.

4.8 Autenticación y autorización

Para identificar a los usuarios usamos un sistema de **tokens de acceso** (JWT). El proceso es el siguiente:

1. El usuario introduce su email y contraseña.

2. El servidor comprueba que la contraseña es correcta (las contraseñas están cifradas en la base de datos, nunca en texto plano).
3. Si todo es correcto, el servidor genera un “token”: una cadena de texto firmada que contiene el identificador del usuario y su tipo (cliente, operario o administrador).
4. La aplicación guarda ese token y lo envía con cada petición al servidor.
5. El servidor verifica la firma del token para asegurarse de que no ha sido manipulado y comprueba que el usuario tiene permiso para lo que pide.

El token caduca a las 8 horas, por lo que el usuario tendrá que volver a iniciar sesión pasado ese tiempo. Para evitar ataques que intentan adivinar contraseñas por fuerza bruta, el servidor limita a 30 los intentos de inicio de sesión por cada cuarto de hora desde la misma dirección IP.

4.9 Swagger / OpenAPI

La documentación interactiva de la API (que permitiría probar cada endpoint desde el navegador) no está implementada actualmente. Todas las rutas disponibles están descritas en la tabla del apartado 4.4 y en las pruebas automáticas del apartado 4.10. La implementación de Swagger está planificada como mejora futura.

4.10 Pruebas de endpoints

Hemos escrito pruebas automáticas con la herramienta **Vitest** que verifican que cada parte del servidor funciona correctamente. En total hay 27 pruebas repartidas en tres archivos:

- **Pruebas de autenticación:** registro, inicio de sesión, rechazo de credenciales incorrectas y bloqueo por exceso de intentos.
- **Pruebas de productos:** creación, edición, borrado, filtros y comprobación de permisos por rol.
- **Pruebas de reservas:** creación de reservas, control de stock insuficiente y, lo más importante, la prueba de concurrencia: dos reservas del mismo producto al mismo tiempo solo puede aprobar una.

5. Desarrollo de la aplicación web

5.1 Diseño inicial en Figma

Ver apartado 2.6. El diseño pasó por tres versiones hasta llegar al estilo actual: minimalista industrial con tipografía condensada y acento en color ocre.

5.2 Estructura del frontend

La página web es una aplicación de una sola página (SPA): se carga una vez y luego cambia el contenido sin recargar el navegador, lo que la hace mucho más rápida. Los archivos principales son:

- **index.html:** la estructura base de la página.
- **app.js:** toda la lógica: el enrutado entre vistas, las llamadas al servidor y la gestión del estado.
- **styles.css:** el sistema de diseño completo, incluyendo el modo oscuro.
- **sw.js:** el Service Worker, que permite que la aplicación funcione sin conexión y pueda instalarse como app de escritorio.

Las vistas disponibles son: login, catálogo de productos, mis reservas, cola de reservas, panel de estadísticas, inventario, gestión de usuarios e importación de datos.

5.3 Integración con la API REST

Para comunicarse con el servidor, la aplicación web usa una función central que añade automáticamente el token de acceso a todas las peticiones. Esta función también detecta cuando el token ha caducado (el servidor devuelve un error 401) y cierra la sesión automáticamente para que el usuario vuelva a identificarse.

5.4 Gestión de sesiones y login

Cuando el usuario inicia sesión, el token de acceso se guarda en el navegador. La próxima vez que abra la aplicación, si el token sigue siendo válido, va directamente a

su vista sin necesidad de volver a escribir la contraseña. Si el token ha caducado o es inválido, se borra y se muestra la pantalla de login.

5.5 CRUD de datos

Las vistas de administración (inventario, categorías, usuarios) muestran todos los registros en una tabla. Desde ahí se puede crear un nuevo elemento (con un botón flotante), editarlo (pulsando directamente en la fila) o eliminarlo. El formulario de creación y el de edición son el mismo, pero al editar aparece pre-rellenado con los datos actuales.

5.6 Funcionalidades según rol

Al iniciar sesión, la aplicación lee el tipo de usuario del token y muestra u oculta las opciones según corresponda. Por ejemplo, el botón de “Dashboard” solo aparece para los administradores. Esta es una mejora de usabilidad, pero la seguridad real está en el servidor, que rechaza cualquier petición no autorizada independientemente de lo que muestre la web.

5.7 Manejo de errores

Cuando algo falla, la aplicación muestra un mensaje emergente (toast) con una descripción clara del problema. Algunos errores tienen tratamiento especial: si el stock se agota justo cuando el usuario intenta reservar, el catálogo se actualiza automáticamente para mostrar el stock real. Los campos de formulario con datos incorrectos se resaltan visualmente.

5.8 Capturas de la aplicación

#	Pantalla	Rol
01	Login	Todos
02	Catálogo con filtros y búsqueda	Cliente / Operario
03	Detalle de producto + modal reserva	Cliente
04	Mis reservas (cliente)	Cliente

#	Pantalla	Rol
05	Cola de reservas (operario)	Operario
06	Panel de estadísticas	Admin
07	Inventario — gestión de productos	Admin
08	Gestión de usuarios	Admin
09	Modo oscuro activo	Todos
10	Botón de descarga de la app Android	Todos

Capturas disponibles en docs/screenshots/app/.

6. Desarrollo de la aplicación móvil

La aplicación móvil está pensada para el operario de almacén. Mientras trabaja, puede consultar en su móvil Android qué reservas están pendientes, confirmar que ha preparado un pedido, registrar la entrega y reportar cualquier incidencia (producto roto, cantidad incorrecta, etc.). Toda esta información queda registrada en el sistema con el nombre del operario y la fecha, lo que permite saber exactamente quién hizo qué y cuándo.

6.1 Diseño inicial en Figma

Las pantallas Android siguen el mismo estilo visual que la web: fondo oscuro, tipografía condensada y acento en color ocre. Este diseño es coherente con el entorno de un almacén, donde la legibilidad en condiciones de poca luz es importante. Los mockups de Figma se exportarán en imagen para incluirlos en el documento final.

6.2 Navegación entre pantallas

La app tiene cuatro pantallas conectadas entre sí:

1. **Pantalla de inicio de sesión:** el operario escribe su email y contraseña. Si ya

inició sesión anteriormente y el token sigue siendo válido, la app salta directamente a la lista de reservas sin pedir credenciales de nuevo.

2. **Lista de reservas:** muestra todas las reservas activas del almacén (las que están pendientes o confirmadas). Se puede filtrar por estado. Cada fila muestra el cliente, el producto, la fecha y el estado actual.
3. **Detalle de una reserva:** al pulsar en una reserva de la lista, se abre esta pantalla con todos los datos del pedido: quién lo pidió, qué producto, cuántas unidades, dónde está ubicado en el almacén y el precio. Desde aquí se puede confirmar el pedido (si está pendiente), marcarlo como entregado (si ya está confirmado) o reportar una incidencia.
4. **Formulario de incidencia:** una pantalla sencilla para indicar qué tipo de problema ha ocurrido (producto roto, cantidad incorrecta, mal estado u otro) y escribir una descripción. La incidencia queda guardada asociada a la reserva y al operario.

6.3 Conexión con la API REST

La app Android se comunica con el mismo servidor que la web. Usamos la librería **Retrofit** para hacer las peticiones de forma sencilla: solo hay que definir qué dirección llamar y qué datos enviar o recibir, y Retrofit se encarga de todo lo demás.

Para que el servidor sepa quién hace cada petición, añadimos automáticamente el token de acceso a todas las llamadas. Esto lo hace un componente llamado **AuthInterceptor** que actúa como intermediario: intercepta cada petición antes de enviarla y añade la cabecera de identificación.

En el modo de desarrollo, la app se conecta al servidor local del ordenador. En la versión publicada, se conecta a la dirección de producción en Railway.

6.4 Persistencia local

Cuando el operario inicia sesión, su token de acceso se guarda en el móvil de forma **cifrada**. Usamos el sistema de seguridad propio de Android (Android Keystore) para que el token no pueda ser leído aunque alguien acceda al almacenamiento interno del teléfono. De esta forma, el operario no tiene que volver a iniciar sesión cada vez que

abra la app.

6.5 Gestión de sesiones

Al abrir la app, el sistema comprueba si hay una sesión guardada. Si la hay y sigue siendo válida, el operario entra directamente en la lista de reservas. Al pulsar “Cerrar sesión”, el token se elimina del móvil y se vuelve a la pantalla de login limpiamente, sin posibilidad de volver atrás con el botón de retroceso.

6.6 Funcionalidades futuras

El flujo principal del operario está completo. Como mejoras para versiones futuras se han identificado:

- **Desbloqueo biométrico:** usar la huella dactilar o el reconocimiento facial para confirmar la identidad antes de mostrar la sesión guardada.
- **Modo sin conexión:** guardar las acciones localmente cuando no hay internet y sincronizarlas cuando la conexión se recupere.
- **Escáner de código de barras:** usar la cámara del móvil para buscar productos escaneando su código, en lugar de escribirlo a mano.
- **Actualización automática de sesión:** detectar cuando el token ha caducado y volver al login automáticamente, sin que el operario tenga que ver un error.

6.7 Capturas de la aplicación

#	Pantalla	Descripción
01	Inicio de sesión	Campos de email y contraseña, botón entrar
02	Lista de reservas	Reservas activas con indicador de estado
03	Detalle de reserva	Datos completos con botones de acción
04	Formulario de incidencia	Tipo de problema y descripción

Capturas a añadir en docs/screenshots/android/ desde dispositivo físico o emulador Android.

7. Despliegue e infraestructura

El sistema está publicado en internet de forma completamente gratuita gracias a **Railway**, una plataforma en la nube pensada para proyectos pequeños y medianos. Railway se encarga de todo lo que normalmente requeriría configurar un servidor propio: el certificado de seguridad HTTPS, el reinicio automático si el servidor se cae, las copias de seguridad de la base de datos y la actualización automática cuando subimos cambios al código.

7.1 Variables de entorno

Para que la misma aplicación funcione tanto en el ordenador de desarrollo como en el servidor en la nube sin cambiar el código, usamos “variables de entorno”: ajustes que se configuran por separado en cada entorno. En Railway estas variables se configuran desde el panel web. Las más importantes son:

Variable	Para qué sirve
Dirección de la base de datos	Para que el servidor sepa dónde está MySQL
Entorno de ejecución	Indica si es desarrollo o producción
Clave secreta	Se usa para firmar los tokens de acceso de forma segura
Tiempo de expiración del token	Cuánto tiempo dura la sesión (8 horas)
Dirección pública	La URL de la aplicación, para el control de acceso entre origen y servidor
Puerto	Railway asigna el puerto automáticamente

En el ordenador local estas variables se guardan en un archivo `.env` que nunca se sube a GitHub.

7.2 Despliegue del backend

Railway detecta automáticamente que es un proyecto Node.js y sabe cómo construirlo y arrancarlo. El proceso es:

1. Cuando subimos código a GitHub, Railway recibe una notificación automática.
2. Railway descarga el código, instala solo las dependencias necesarias para producción y arranca el servidor.
3. Si el servidor no responde correctamente en la dirección de salud (/api/health), Railway lo reinicia automáticamente hasta 5 veces.

7.3 Base de datos remota

Railway incluye un servicio de MySQL que funciona como una base de datos independiente dentro del mismo proyecto. Al conectarlos, Railway proporciona automáticamente la dirección y credenciales de conexión sin que tengamos que configurar nada manualmente.

La primera vez que el servidor arranca en producción, detecta que la base de datos está vacía y crea todas las tablas e inserta los datos de prueba. En los arranques siguientes, como las tablas ya existen, no se toca nada y los datos guardados permanecen intactos.

7.4 Hosting y servicios cloud

Componente	Servicio	Ventajas
Servidor Node.js	Railway	Reinicio automático, sin configuración de servidor
Base de datos MySQL	Railway	Copias de seguridad diarias, sin administración
Página web	Servida por el propio servidor	Sin coste adicional ni configuración extra
Certificado HTTPS	Gestionado por Railway	Seguridad automática sin renovaciones manuales

7.5 Git y GitHub

Todo el código del proyecto está en un repositorio de GitHub. Usamos Git para registrar cada cambio: qué se modificó, cuándo y por qué. Los mensajes de cada cambio siguen una convención para que sea fácil ver el historial: `feat`: para nuevas funcionalidades, `fix`: para correcciones, `docs`: para documentación y `chore`: para tareas de mantenimiento.

Cada vez que se sube un cambio a la rama principal (`main`), Railway lo detecta y despliega la nueva versión automáticamente en cuestión de minutos.

7.6 GitHub Actions / CI/CD

El despliegue automático desde GitHub a Railway ya está activo. Las comprobaciones de calidad del código (el linter que detecta errores de estilo y los tests automáticos) se ejecutan en el ordenador antes de subir cambios. Como mejora futura está planificado configurar estas comprobaciones también en GitHub Actions, para que fallen automáticamente si alguien sube código con errores.

7.7 Acceso público al sistema

Recurso	Dirección
Aplicación web	https://tfgdam-production.up.railway.app/
API REST	https://tfgdam-production.up.railway.app/api/
Comprobación de estado	https://tfgdam-production.up.railway.app/api/health

8. Seguridad

8.1 Autenticación

Las contraseñas nunca se guardan en texto plano en la base de datos. Se usa un algoritmo de cifrado llamado **bcrypt** que transforma la contraseña en una cadena

irreversible. Cuando el usuario inicia sesión, se compara la contraseña introducida con esa cadena, sin necesidad de descifrarla.

Los tokens de acceso están firmados digitalmente con una clave secreta que se genera automáticamente al primer arranque de la aplicación y nunca se sube a GitHub. Esto significa que aunque alguien consiga un token, no puede modificar su contenido sin que el servidor lo detecte.

Para evitar que alguien intente adivinar contraseñas probando miles de combinaciones, el servidor limita a 30 los intentos de inicio de sesión cada cuarto de hora desde la misma IP.

8.2 Autorización

Cada operación del servidor comprueba explícitamente qué tipo de usuario tiene permiso para realizarla. El tipo de usuario viaja dentro del token firmado, por lo que no puede ser modificado por el usuario. La página web oculta las opciones que no corresponden a cada rol, pero esto es solo para mejorar la experiencia: la barrera real de seguridad siempre está en el servidor.

8.3 Protección de datos

Hemos aplicado varias medidas para proteger la aplicación de los ataques más comunes:

- **Inyección SQL:** todas las consultas a la base de datos usan parámetros separados, lo que impide que un atacante pueda manipular las consultas introduciendo código SQL en los formularios.
- **XSS (Cross-Site Scripting):** el contenido dinámico se inserta en la página como texto plano, nunca como código HTML. Esto evita que un atacante pueda inyectar código malicioso en la página.
- **HTTPS obligatorio:** en producción todas las comunicaciones van cifradas, lo que impide que alguien intercepte los tokens de acceso en tránsito.
- **Datos personales mínimos:** solo guardamos el email y el nombre del usuario. No almacenamos datos de pago, DNI ni dirección, lo que reduce el impacto de

una posible brecha de datos.

8.4 Buenas prácticas aplicadas

- Revisión periódica de las dependencias para detectar vulnerabilidades conocidas.
 - El archivo `.env` con las claves secretas está excluido del repositorio de GitHub.
 - No se usa ninguna función que ejecute código dinámico (como `eval`), que es una fuente común de vulnerabilidades.
 - Los registros del servidor (logs) se guardan con la información necesaria para depurar problemas, sin incluir contraseñas ni datos sensibles.
-

9. Inteligencia Artificial aplicada al proyecto

9.1 Herramientas utilizadas

Durante el desarrollo hemos utilizado tres herramientas de inteligencia artificial:

- **Claude Code** (de Anthropic): fue el asistente principal. Lo usamos para diseñar la arquitectura, generar borradores de código, depurar errores y redactar documentación.
- **GitHub Copilot**: integrado en el editor de código, nos sugirió completaciones de código mientras escribíamos.
- **ChatGPT**: para consultas puntuales cuando necesitábamos una segunda opinión o una explicación rápida de algún concepto.

9.2 Uso realizado

Usamos la IA como un compañero de programación, no como un sustituto. La diferencia es importante: el compañero propone, nosotros decidimos.

- **Para escribir código**: la IA generó borradores iniciales de las rutas del servidor, las validaciones y la capa de red de la app Android. Todos fueron revisados, probados y adaptados antes de incluirlos en el proyecto.

- **Para el diseño visual:** cuando el sistema de diseño de la segunda versión resultó demasiado recargado, usamos la IA para explorar alternativas más limpias hasta llegar a la versión actual.
- **Para la documentación:** la IA ayudó a redactar el borrador inicial de esta memoria, el README del proyecto y el registro de cambios. Todo fue revisado y corregido manualmente.
- **Para resolver errores:** cuando la aplicación tenía comportamientos extraños (el Service Worker que no se actualizaba, los problemas de concurrencia en las reservas, la configuración de MySQL en diferentes máquinas), la IA ayudó a diagnosticar las causas.
- **Para la base de datos:** el esquema inicial de las tablas y los índices se diseñaron con ayuda de la IA, y luego se validaron con datos reales.
- **Para las pruebas:** la IA generó la estructura inicial de los tests; los casos límite más interesantes (concurrencia, comprobación de permisos) los ampliamos nosotros manualmente.

9.3 Validación y revisión

Cada vez que la IA generaba algo, lo hacíamos pasar por tres filtros antes de usarlo:

1. **Lectura crítica:** ¿tiene sentido lo que propone? ¿encaja con el resto del proyecto?
2. **Pruebas locales:** ejecutar el código y comprobar que funciona en los casos normales y en los casos límite.
3. **Ajuste de estilo:** el código generado por IA tiende a ser más formal o a usar convenciones distintas a las del resto del proyecto, así que siempre hay que homogeneizarlo.

La IA también tiene limitaciones que aprendimos a gestionar: a veces propone soluciones innecesariamente complejas, a veces usa versiones antiguas de librerías, y pierde el contexto del proyecto en conversaciones largas.

Conclusión: la IA fue una herramienta muy útil para acelerar las partes más repetitivas del desarrollo, pero la responsabilidad sobre las decisiones de diseño y la calidad del código final es nuestra.

10. Pruebas y validación

10.1 Pruebas funcionales

Hemos realizado dos tipos de pruebas:

Pruebas automáticas: escritas con Vitest y Supertest, ejecutan 27 comprobaciones sobre el servidor sin necesidad de abrir el navegador. Todas pasan correctamente. Cubren aproximadamente el 65% de las líneas de código y el 100% de los flujos críticos (login, reservas, concurrencia, permisos).

Pruebas manuales: hemos probado la instalación de la PWA en Chrome para escritorio y en Android, el comportamiento de la app cuando no hay conexión, el modo claro y oscuro, el diseño responsive en pantallas desde 320px (móvil pequeño) hasta 1500px (pantalla grande) y la impresión del albarán en A4.

10.2 Casos de prueba

ID	Qué probamos	Resultado esperado	Estado
TC-01	Iniciar sesión con datos correctos	El servidor devuelve un token y carga la vista	OK
TC-02	Iniciar sesión con contraseña incorrecta	El servidor devuelve error 401	OK
TC-03	Registrarse con un email que ya existe	El servidor devuelve error 409	OK
TC-04	Reservar un producto con stock suficiente	Se crea la reserva y se reduce el stock disponible	OK
TC-05	Reservar con stock insuficiente	El servidor devuelve error 409	OK
TC-06	Dos reservas del mismo producto al mismo tiempo	Solo una se acepta, la otra recibe error de stock	OK

ID	Qué probamos	Resultado esperado	Estado
TC-07	Un cliente intenta borrar un producto	El servidor devuelve error 403 (sin permiso)	OK
TC-08	El operario entrega una reserva	El stock se descuenta y el estado cambia a “entregada”	OK
TC-09	Usar un token de acceso caducado	El servidor devuelve error 401	OK
TC-10	Superar el límite de intentos de inicio de sesión	El servidor devuelve error 429 y bloquea temporalmente	OK

10.3 Resultados obtenidos

Todas las pruebas automáticas pasan correctamente. Los flujos principales han sido verificados tanto en el entorno de desarrollo local como en el servidor de producción en Railway (<https://tfgdam-production.up.railway.app>).

10.4 Errores encontrados y soluciones

Durante el desarrollo encontramos varios problemas que resolvimos:

- **Conflicto de puerto con MySQL:** en algunos ordenadores, el puerto 3306 de MySQL ya está en uso por otra instancia. El script de arranque detecta este caso y cambia automáticamente al puerto 3307.
- **Reservas duplicadas:** al principio era posible que dos usuarios reservaran el mismo producto al mismo tiempo y juntos superaran el stock. Lo resolvimos bloqueando temporalmente el registro del producto en la base de datos durante la transacción de reserva.
- **Token que cambia al reiniciar:** el token de acceso se firmaba con una clave generada al azar en cada arranque, lo que invalidaba todas las sesiones activas cada vez que se reiniciaba el servidor. Lo solucionamos guardando la clave en el archivo `.env` la primera vez que se genera.
- **PWA “pegada” en versión vieja:** cuando actualizábamos la aplicación, algunos

usuarios seguían viendo la versión anterior porque el navegador la había guardado en caché. Lo resolvimos con un Service Worker especial que detecta las cachés antiguas y las elimina automáticamente.

- **Diseño demasiado recargado:** la segunda versión del diseño resultó visualmente saturada. Hicimos un rediseño completo a la tercera versión, más limpia y legible.
-

11. Conclusiones

11.1 Resultados obtenidos

El proyecto ha producido un sistema completo y funcional:

- Una **página web** instalable como aplicación de escritorio, con catálogo de 500 productos, gestión completa de reservas, panel de estadísticas, importación y exportación de datos en CSV, e instaladores para Windows.
- Una **aplicación Android nativa** que permite al operario gestionar las reservas del almacén desde su móvil, con sesión cifrada y cuatro pantallas funcionales.
- Un **despliegue en producción** accesible públicamente desde cualquier dispositivo con conexión a internet.
- Un sistema de **pruebas automáticas** que verifica los flujos más importantes, incluida la gestión de reservas concurrentes.

11.2 Problemas encontrados

Los retos más importantes que tuvimos que superar:

- **La concurrencia:** entender por qué simplemente “comprobar el stock antes de reservar” no es suficiente cuando dos usuarios lo hacen al mismo tiempo fue el problema técnico más interesante del proyecto. Requirió estudiar cómo funcionan las transacciones en bases de datos y el concepto de bloqueo pesimista.
- **Iterar el diseño:** llegar a un diseño que funcione visualmente llevó más tiempo del esperado. Descartamos dos versiones completas antes de dar con el equilibrio

correcto.

- **Compatibilidad entre máquinas:** configurar MySQL de forma que funcione igual en distintos ordenadores (con versiones diferentes, puertos ocupados, etc.) resultó más complicado de lo previsto.
- **La caché de la PWA:** aprendimos que una actualización mal implementada puede dejar a los usuarios atascados en una versión antigua de la aplicación durante días.

11.3 Mejoras futuras

Las principales líneas de mejora identificadas para versiones futuras:

- **App Android mejorada:** autenticación biométrica (huella dactilar), funcionamiento sin conexión y escáner de códigos de barras.
- **Documentación de la API:** implementar una interfaz interactiva que permita explorar y probar los endpoints del servidor desde el navegador.
- **Accesibilidad completa:** cumplir el estándar WCAG AA en todos los elementos de la interfaz.
- **Automatización de pruebas en GitHub:** ejecutar los tests automáticamente en cada subida de código, no solo en local.
- **Funcionalidades adicionales:** subida de imágenes de productos, notificaciones push, recuperación de contraseña por email y soporte para múltiples almacenes.

11.4 Valoración personal

Este proyecto nos ha permitido conectar todo lo aprendido durante el ciclo en un producto real que resuelve un problema concreto. No ha sido solo aplicar conocimientos: ha sido tomar decisiones con consecuencias reales y aprender de los errores.

Lo más valioso ha sido la práctica de iterar: diseñar, probar, descubrir que no funciona del todo bien y volver a empezar sin desanimarse. También hemos aprendido a trabajar con inteligencia artificial de forma crítica: es una herramienta que multiplica la velocidad, pero que exige más responsabilidad, no menos, porque hay que entender lo que propone para poder validarlo.

Los dos objetivos más ambiciosos del proyecto — publicar el sistema en producción y desarrollar la app Android — están cumplidos. El sistema corre en Railway y el operario puede gestionar las reservas desde su móvil. Queda trabajo por hacer, pero la base es sólida.

12. Bibliografía

Documentación oficial consultada: - Node.js: <https://nodejs.org/docs> - Express: <https://expressjs.com> - MySQL 8: <https://dev.mysql.com/doc/refman/8.0/en/> - MDN Web Docs (referencia HTML, CSS y JavaScript): <https://developer.mozilla.org> - Jetpack Compose (Android): <https://developer.android.com/jetpack/compose> - Retrofit (librería de red Android): <https://square.github.io/retrofit/> - JWT — Introducción a los tokens de acceso: <https://jwt.io/introduction>

Libros consultados: - Martin Kleppmann, Diseño de aplicaciones intensivas en datos — consultado para entender las transacciones y la concurrencia en bases de datos. - Martin Fowler, Refactoring (2.^a edición) — para mejorar la estructura del código sin cambiar su comportamiento.

Recursos web: - Railway (documentación de despliegue): <https://docs.railway.com> - WCAG 2.1 — Guía de accesibilidad web: <https://www.w3.org/WAI/WCAG21/quickref/> - OWASP Top 10 — Los diez riesgos de seguridad más comunes: <https://owasp.org/Top10/>

Herramientas utilizadas: Git, GitHub, Visual Studio Code, MySQL Workbench, Figma, Postman, Vitest, ESLint, Claude Code, GitHub Copilot, Inno Setup.

13. Anexos

Anexo A — Manual de instalación

Opción 1 — Instalador Windows: Ejecutar el archivo Stockly-Setup.exe. El instalador comprueba si Node.js y MySQL están instalados y los instala si es necesario, crea

la base de datos, arranca el servidor y abre la aplicación en el navegador.

Opción 2 — Instalación manual: 1. Clonar el repositorio desde GitHub. 2. Ejecutar `start.bat` en Windows o seguir las instrucciones del `README.md` en otros sistemas. 3. El script inicializa la base de datos si no existe y arranca el servidor en `http://localhost:3001`.

Usuarios de prueba (contraseña para todos: `password123`): - `adrian@tfg.local` — Administrador - `laura@tfg.local` — Operario - `marcos@tfg.local` — Cliente

Anexo B — Manual de usuario

Como cliente: 1. Ir al catálogo de productos. 2. Buscar el producto que necesitas y abrirlo. 3. Indicar cuántas unidades quieres y pulsar “Reservar”. 4. En “Mis reservas” puedes ver el estado de tus pedidos.

Como operario: 1. En la pestaña “Reservas” verás todos los pedidos pendientes. 2. Cuando prepares un pedido, pulsa “Confirmar”. 3. Cuando lo entregues al cliente, pulsa “Entregar” e imprime el albarán si es necesario. 4. Si hay algún problema con el pedido, usa el botón de incidencia para registrarlo.

Como administrador: - En “Inventario” puedes crear, editar y eliminar productos. - En “Dashboard” tienes un resumen del estado del almacén. - En “Usuarios” puedes gestionar las cuentas del equipo. - Puedes importar productos en lote desde un archivo CSV.

Manual detallado con capturas paso a paso disponible en `docs/manual-usuario.md`.

Anexo C — Enlaces

- **Código fuente en GitHub:** <https://github.com/husslesnake/TFGDAM>
- **Aplicación web en producción:** <https://tfgdam-production.up.railway.app/>
- **Swagger / OpenAPI:** pendiente de implementar.
- **Vídeo demostración:** pendiente de grabar — flujo completo: login → reserva → confirmar → entregar → incidencia.

Anexo D — Fragmentos de código relevantes

D.1 Reserva con control de concurrencia

El punto más delicado del proyecto fue garantizar que dos usuarios no pudieran reservar más stock del disponible aunque lo intentaran exactamente al mismo tiempo. La solución fue usar una **transacción con bloqueo**: cuando el servidor empieza a procesar una reserva, bloquea el registro del producto en la base de datos para que ninguna otra reserva pueda leerlo o modificarlo hasta que la primera termine. Si no hay suficiente stock, se cancela la operación y se libera el bloqueo. Si hay stock, se crea la reserva, se actualiza el stock y se libera el bloqueo. Todo esto ocurre en milisegundos.

```
// backend/src/routes/reservas.js - POST /api/reservas
router.post('/', authRequired, async (req, res) => {
  const { producto_id, cantidad } = req.body || {};

  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    // Bloquear el producto mientras procesamos la reserva
    const [[prod]] = await conn.query(
      'SELECT stock, stock_reservado FROM productos WHERE id = ? AND activo = 1'
      [producto_id]
    );
    const disponible = prod.stock - prod.stock_reservado;
    if (cantidad > disponible) {
      await conn.rollback();
      return res.status(409).json({ error: `Stock insuficiente (disponible: ${disponible})` });
    }
    // Actualizar stock, crear reserva y registrar movimiento
    await conn.query('UPDATE productos SET stock_reservado = stock_reservado + ? WHERE id = ?'
      [cantidad, producto_id]);
    const [ins] = await conn.query(

```

```

        'INSERT INTO reservas (usuario_id, producto_id, cantidad) VALUES (?, ?, ?)'
        [req.user.id, producto_id, cantidad]);
    await conn.commit();
    res.status(201).json({ id: ins.insertId });
} catch (e) {
    await conn.rollback();
    res.status(500).json({ error: e.message });
} finally {
    conn.release();
}
});

```

D.2 Service Worker de autodesregistro

Durante el desarrollo subimos una versión de la aplicación que guardaba en caché la dirección del servidor de desarrollo (localhost). Cuando desplegamos en producción, los usuarios que habían visitado la página antes seguían viendo la versión en caché con la dirección incorrecta. Para solucionarlo creamos un Service Worker especial que, al activarse, elimina todas las cachés antiguas y se desregistra a sí mismo, forzando al navegador a descargar la versión nueva limpia.

```

// frontend/sw.js – Service Worker de autodesregistro
self.addEventListener('install', () => self.skipWaiting());

self.addEventListener('activate', e => {
    e.waitUntil(async () => {
        const keys = await caches.keys();
        await Promise.all(keys.map(k => caches.delete(k)));
        await self.registration.unregister();
        const clients = await self.clients.matchAll({ type: 'window' });
        clients.forEach(c => c.navigate(c.url));
    })();
});

```

```
self.addEventListener('fetch', () => {}); // sin caché – red directa
```

D.3 Capa de red Android

La app Android usa Retrofit para comunicarse con el servidor. Definimos los endpoints como métodos de una interfaz, y Retrofit se encarga automáticamente de construir las peticiones HTTP, añadir el token de acceso y deserializar las respuestas JSON a objetos Kotlin.

```
// mobile-android/.../data/StocklyApi.kt
interface StocklyApi {
    @POST("api/auth/login")
    suspend fun login(@Body body: LoginRequest): LoginResponse

    @GET("api/reservas")
    suspend fun reservas(@Query("activas") activas: Int? = null): List<ReservaListItem>

    @GET("api/reservas/{id}")
    suspend fun reserva(@Path("id") id: Int): ReservaDetalle

    @PATCH("api/reservas/{id}/estado")
    suspend fun cambiarEstado(@Path("id") id: Int, @Body body: EstadoRequest): OkResponse

    @POST("api/reservas/{id}/incidencias")
    suspend fun crearIncidencia(@Path("id") id: Int, @Body body: CrearIncidenciaRequest)
}
```

Fin del documento.